LAPORAN
TUGAS KELOMPOK MOBILE

Oleh:

Kelompok 8

Kevin Maleakhi	**2210817210031**
Muhammad Aufa Fitrianda	**2310817210013**
Muhammad Ilham	**2410817310017**
Oktavia Laila Qodari	**2410817120013**

PROGRAM STUDI S1 TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

BANJARMASIN

2026

BAB I: PENDAHULUAN

1.1 Latar Belakang

Pengembangan aplikasi seluler telah mengalami perkembangan yang sangat pesat. Salah satu kemajuan terpenting dalam ekosistem Android adalah pergeseran dari antarmuka pengguna (UI) yang dibangun menggunakan format XML imperatif menuju paradigma deklaratif kontemporer. Jetpack Compose, sebuah toolkit kontemporer yang dibuat oleh Google untuk membangun UI Android secara natif, berkontribusi pada pembentukan paradigma baru ini.

Aplikasi prototipe yang dikembangkan dalam proyek Kelompok 4 ini menunjukkan kemampuan Jetpack Compose untuk menangani tata letak dinamis, navigasi antar halaman, dan integrasi langsung dengan layanan sistem perangkat keras seperti sensor baterai dan status konektivitas jaringan.

1.2 Tujuan Laporan

1. Membedah struktur dan hierarki kode dari aplikasi Jetpack Compose.

2. Menganalisis bagaimana data statis (Sejarah Android) dirender dalam komponen List yang dapat digulir (scrollable).

3. Mengevaluasi metode pengambilan data dinamis dari perangkat keras (Baterai, Jaringan, dan OS) menggunakan layanan sistem bawaan Android.

4. Menyediakan dokumentasi teknis yang komprehensif terkait komponen UI, Modifiers, dan State Management.

1.3 Ruang Lingkup

Analisis dalam laporan ini dibatasi pada berkas utama (MainActivity.kt) yang berisi antarmuka tunggal dengan dua halaman (Page 0: Sejarah Android, Page 1: Arsitektur & Informasi Perangkat). Laporan ini membahas kode dari perspektif frontend (UI/UX) dan backend (pengambilan data perangkat).

BAB II: TINJAUAN PUSTAKA

2.1 Konsep Dasar Jetpack Compose

Jetpack Compose memudahkan dan mempercepat pengembangan UI di Android. Memungkinkan pengembang mendeskripsikan bagaimana bentuk UI pada suatu kondisi (State), alih-alih memanipulasi tampilan (views) secara manual melalui findViewById dan mengubah statusnya. Compose secara cerdas akan me-render ulang (recompose) bagian UI yang terdampak saja saat kondisi berubah.Seluruh antarmuka pengguna (UI) didefinisikan dalam fungsi-fungsi yang dianotasi dengan @Composable dalam kode aplikasi ini. Ini menunjukkan bahwa fungsi tersebut memiliki kemampuan untuk mengubah data menjadi hierarki UI.

2.2 Arsiktektur Sistem Android

Aplikasi ini menampilkan susunan lapisan Arsitektur Android. Pemahaman teoritis mengenai lapisan ini penting karena memengaruhi cara aplikasi berinteraksi dengan perangkat keras keras. :

1. Linux Kernel: Fondasi paling bawah yang mengatur memori, proses, dan driver perangkat keras.

2. Hardware Abstraction Layer (HAL): Menjembatani antara perangkat keras spesifik dan framework Java.

3. Libraries & Android Runtime (ART): Berisi pustaka inti C/C++ (seperti SQLite, OpenGL) dan mesin virtual yang menjalankan kode aplikasi.

4. Java API Framework: Kumpulan API tingkat tinggi (termasuk BatteryManager dan ConnectivityManager yang digunakan dalam aplikasi ini).

5. Applications: Lapisan teratas tempat aplikasi ini, dan aplikasi lainnya (seperti kalender, browser), berjalan.

BAB III: ANALISIS STRUKTUR UTAMA APLIKASI

3.1 Konfigurasi Activity Utama (MainActivity)

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { MaterialTheme { MainApp() } }
    }
}
```

Gambar 3.1 Konfigurasi Activity Utama

Jika aplikasi Android saat ini sepenuhnya bergantung pada Compose, mereka menggunakan Aktivitas Komponen daripada AppCompatActivity. Hirakhi Compose dimasukkan ke dalam aktivitas melalui fungsi setContent. Setiap komponen, termasuk teks dan tombol, akan mewarisi panduan desain Material dengan menggunakan Tema Material.

3.2 Sistem Navigasi Pager (MainApp)

Fungsi @Composable fun MainApp() bertindak sebagai wadah (container) utama aplikasi.

```
val pagerState = rememberPagerState { 2 }
val labels = listOf("SEJARAH ANDROID", "INFO & ARSITEKTUR")
```

Gambar 3.2 Pengelolaan State Pager

Aplikasi menggunakan pager horizontal dua halaman. Fungsi RememberPagerState sangat penting karena memungkinkan Compose untuk menyimpan state memori dengan kata kunci Remember. Jika ini tidak digunakan, posisi halaman akan diatur ulang ke nol setiap kali pengguna menggeser layar.

```
Row(Modifier.height(30.dp).fillMaxWidth(), horizontalArrangement = Arrangement.Center, verticalAlignment = Alignment.
    repeat( times = 2) { iteration ->
        Box(Modifier.padding( all = 4.dp).size(10.dp).background( color = if (pagerState.currentPage == iteration)
    }
}
```

Gambar 3.3 Indikator Halaman (Page Indicator):

Kode ini dengan cermat membuat indikator dot, yang sering ditemukan di aplikasi kontemporer. Ia menggunakan repeat repeat(2). Logika kondisional if (pagerState.currentPage = iteration) memastikan bahwa titik yang aktif berwarna hitam dan titik yang tidak aktif berwarna abu-abu terang. Ini juga dibentuk dalam bentuk lingkaran.

```
HorizontalPager(pagerState, Modifier.weight(1f)) { page ->
    if (page == 0) HistoryContent() else DeviceInfoContent()
}
```

Gambar 3.4 Logika Pemanggilan Konten

Setelah menampilkan judul dan indikator titik, HorizontalPager akan mengambil seluruh ruang layar yang tersedia dengan menggunakan Modifier.weight(1f). Apakah layar "Sejarah" atau "InfoPerangkat" harus ditampilkan berdasarkan indeks halaman saat ini ditentukan oleh blok kondisional if-else.

BAB IV: IMPLEMENTASI KONTEN SEJARAH ANDROID

Fungsi Halaman pertama ditampilkan oleh fungsi @Composable fun HistoryContent(). Ini adalah contoh ideal dari cara Compose mengubah data berbasis daftar (List atau Map) menjadi komponen visual.

4.1 Struktur Data Historis

Data disimpan dalam struktur mapOf (Pasangan Kunci-Nilai / Key-Value Pair):

```
val events = mapOf(
    "2003" to "Android Inc. didirikan di Palo Alto.",
    "2005" to "Google mengakuisisi Android Inc.",
    "2007" to "Open Handset Alliance diumumkan.",
    "2008" to "Ponsel Android pertama (HTC Dream) dirilis.",
    "Sekarang" to "OS paling populer di dunia saat ini."
```

Gambar 4.1 Struktur Data Historis

Penggunaan Map di sini sangat efisien untuk memetakan Tahun/Masa (sebagai kunci) ke Peristiwa (sebagai nilai) karena ukurannya kecil dan statis.

4.2 Menampilkan Daftar yang Dapat DiScroll (Scrollable List)

```
Column(Modifier.fillMaxSize().verticalScroll( state = rememberScrollState())) {
    events.forEach { (year, text) ->
        Card(Modifier.fillMaxWidth().padding(vertical = 4.dp)) {
            Row(Modifier.padding( all = 16.dp)) {
                // Lebar 95dp supaya tulisan "Sekarang" aman tidak terpotong
                Text(year, Modifier.width(95.dp), fontWeight = FontWeight.Bold, color = Color(0xFF2196F3))
                Text(text, modifier = Modifier.weight(1f))
```

Gambar 4.2 Menampilkan Daftar yang Dapat di scroll(Scrollable List)

Kode ini menggunakan Kolom biasa dikombinasikan dengan scroll vertical(rememberScrollState()) daripada LazyColumn, yang disarankan untuk daftar rak dengan hanya beberapa rak atau ratusan item. Mengingat bahwa hanya ada lima data, keputusan teknis ini sangat tepat. Akan ada overhead kinerja yang tidak diperlukan jika Anda menggunakan LazyColumn untuk lima item.

4.3 Desain antarmuka Kartu

Di dalam Perulagan foreach, Setiap iterasi menghasilkan card terdapat Penyusaian Khusus pada komponen teks : text (Year, Modifier.width(95.dp), fontWeight = FontWeight.Bold, color = Color(0xFF2196F3)). Developer telah memberikan lebar 95.dp untuk teks di sisi kiri. "Lebar 95dp supaya tulisan "Sekarang" aman tidak terpotong," kata komentar kode. Ini adalah teknik rekayasa antarmuka yang sangat baik yang menghindari layout shift, atau pergeseran tata letak, di mana elemen antarmuka berantakan karena panjang karakter yang berbeda di kolom kiri. Warna biru material standar yang digunakan (0xFF2196F3) memberikan kontras visual yang baik terhadap teks biasa di sebelahnya.

BAB V: IMPLEMENTASI INFO PERANGKAT DAN ARSITEKTUR

Fungsi Composable fun DeviceInfoContent() merupakan bagian paling kompleks secara teknis dari aplikasi ini, karena melibatkan pengambilan Konteks (Context) dan integrasi dengan layanan perangkat keras.

5.1 Visualisasi Arsitektur Sistem

Sama seperti komponen Sejarah, lapisan arsitektur disimpan dalam struktur data statis, namun kali ini menggunakan kombinasi List dan Pair untuk memetakan nama lapisan dengan representasi warna heksadesimalnya.

1. "Applications" direpresentasikan dengan hijau (0xFF4CAF50).
2. "Java API Framework" dengan biru (0xFF2196F3).
3. "Libraries & Runtime" dengan oranye (0xFFFF9800).
4. "Hardware Abstraction Layer" dengan merah (0xFFF44336).
5. "Linux Kernel" dengan ungu (0xFF673AB7).

Warna-warna ini kemudian dimasukan langsung ke dalam parameter CardDefaults.cardColors(color). Penggunaan kotak (Box) di dalam Kartu dengan Alignment.Center memastikan teks berada tepat di tengah-tengah kartu warna, mensimulasikan diagram blok arsitektur.

5.2 Pengambilan Konteks Sistem Lokal

val ctx = LocalContext.current. Di Jetpack Compose, UI tidak memiliki akses langsung ke sistem Android utama. LocalContext adalah CompositionLocal bawaan yang memungkinkan fungsi bertipe @Composable untuk mendapatkan Context dari Activity tempat aplikasi berjalan. Context ini mutlak diperlukan untuk mengakses layanan perangkat keras keras di langkah selanjutnya.

5.3 Pemantauan Kapasitas Baterai (BatteryManager API)

```
val battery = try {
    val bm = ctx.getSystemService( name = Context.BATTERY_SERVICE) as BatteryManager
    bm.getIntProperty( id = BatteryManager.BATTERY_PROPERTY_CAPACITY)
} catch (_: Exception) { 0 }
```

Gambar 5.1 Pemantauan Kapasitas Baterai (batteryManager Api)

Analisis Metode:

1. Aplikasi membutuhkan BATTERY_SERVICE dari sistem. Ini adalah fitur yang termasuk dalam Java API Framework Android.
2. Penggunaan tipe pemeran eksplisit (as BatteryManager) mengubah layanan generik menjadi objek spesifik baterai.

3. Dengan menggunakan argumen BATTERY_PROPERTY_CAPACITY, getIntProperty akan mengembalikan nilai integer mulai dari 0 hingga 100 yang menunjukkan persentase sisa daya.

4. Keamanan: Untuk situasi ini, penggunaan blok try-catch sangat penting. Pemanggilan hardware dapat gagal pada beberapa sistem emulator atau perangkat yang dimodifikasi secara ekstrem. Blok catch secara tiba-tiba mencegah aplikasi crash (Force Close) dan mengembalikan nilai aman (0).

5.4 Pemantauan Jaringan (ConnectivityManager API)

```
val net = try {
    val cm = ctx.getSystemService( name = Context.CONNECTIVITY_SERVICE) as ConnectivityManager
    val nw = cm.activeNetwork
    val cap = if (nw != null) cm.getNetworkCapabilities( network = nw) else null
    when {
        cap == null -> "Offline"
        cap.hasTransport( transportType = NetworkCapabilities.TRANSPORT_WIFI) -> "Wi-Fi"
        cap.hasTransport( transportType = NetworkCapabilities.TRANSPORT_CELLULAR) -> "Seluler"
        else -> "Terhubung"
    }
} catch (_: Exception) { "Unknown" }
```

Gambar 5.2 Pemantauan Jaringan (connectivityManager Api)

Analisis Metode:

1. Aplikasi menginisialisasi CONNECTIVITY_SERVICE.

2. cm.activeNetwork memanggil jaringan default yang saat ini sedang mentransmisikan data.

3. cm.getNetworkCapabilities(nw) memeriksa jaringan tersebut, misalnya bandwidth, aturan metered, dan jenis transportasinya.

4. Pola struktur when (pengganti switch-case di Java) bertindak sebagai penyaring cerdas:

- Jika kapabilitas nihil (null), berarti perangkat "Offline".

- Fungsi hasTransport mengecek medium fisiknya. Jika mediumnya TRANSPORT_WIFI, aplikasi mengenali koneksi "Wi-Fi". Jika TRANSPORT_CELLULAR, dikenali sebagai "Seluler" (LTE/4G/5G).

5. Sama seperti sensor baterai, modul ini dibungkus dalam try-catch untuk stabilitas produksi.

5.5 Render Data Perangkat Keras dan OS

Setelah semua variabel perangkat keras dan sistem operasi diikat dalam memori (battery, net, Build.MODEL untuk nama ponsel, dan Build.VERSION.RELEASE untuk versi Android), data tersebut dirangkum dalam List dan dirender menggunakan perulangan forEach. Desainnya memanfaatkan Row dengan modifier lebar statis (100.dp) pada label (seperti "Model:", "Baterai:") untuk memastikan kolom nilai (value) sejajar secara vertikal dengan rapi di seluruh baris.

BAB VI: KESIMPULAN

Analisa kode sumber MainActivity.kt menunjukkan implementasi aplikasi Android yang sangat terstruktur. Pengembang telah menunjukkan dengan jelas bahwa Jetpack Compose memungkinkan integrasi interaksi dengan perangkat keras tingkat menengah, seperti deteksi Wi-Fi dan persentase baterai, ke dalam alur UI dengan cepat dan mudah tanpa memerlukan callback yang rumit atau penerima broadcast yang membebani memori. memori. Memahami siklus hidup aplikasi dan toleransi kesalahan (fault tolerance) dalam pengembangan perangkat lunak kontemporer ditunjukkan dengan penggunaan manajemen state pada HorizontalPager, pengaturan lebar khusus untuk menghindari masalah layout shift, dan pembungkusan layanan API Android inti dengan blok try-catch.